

Vision fine-tuning GPT-4o on a custom dataset

Learn to vision fine-tune GPT-4o on a custom dataset, with evaluation and tracking.

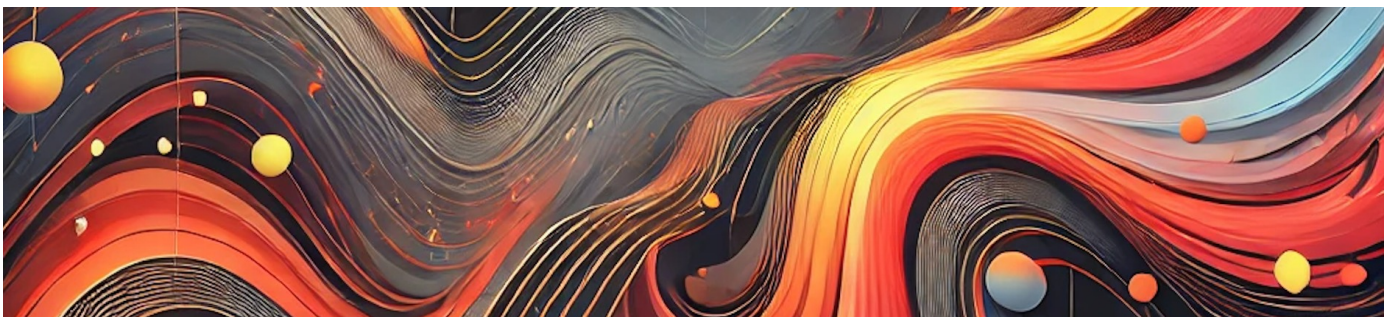
Brett Young



Created on October 7 | Last edited on October 17

In this tutorial, we'll explore how to vision fine-tune GPT-4o using the [OpenAI](#) API and evaluate its performance with [W&B Weave](#)'s evaluation tools. Our focus will be on a practical use case involving a multimodal dataset called ChartQA, which contains question-answer pairs based on visual data from charts and tables. By leveraging OpenAI's capabilities, we can create a custom model that is fine-tuned to handle this specific data, allowing for more precise and context-aware responses.

You can follow along in the code below, or visit our Colab, if you want to jump right in.

[Open in Colab](#)

Establishing a Baseline



By clicking "Accept All Cookies", you agree to the storing of cookies on your device to enhance site navigation, analyze site usage, and assist in our marketing efforts. 

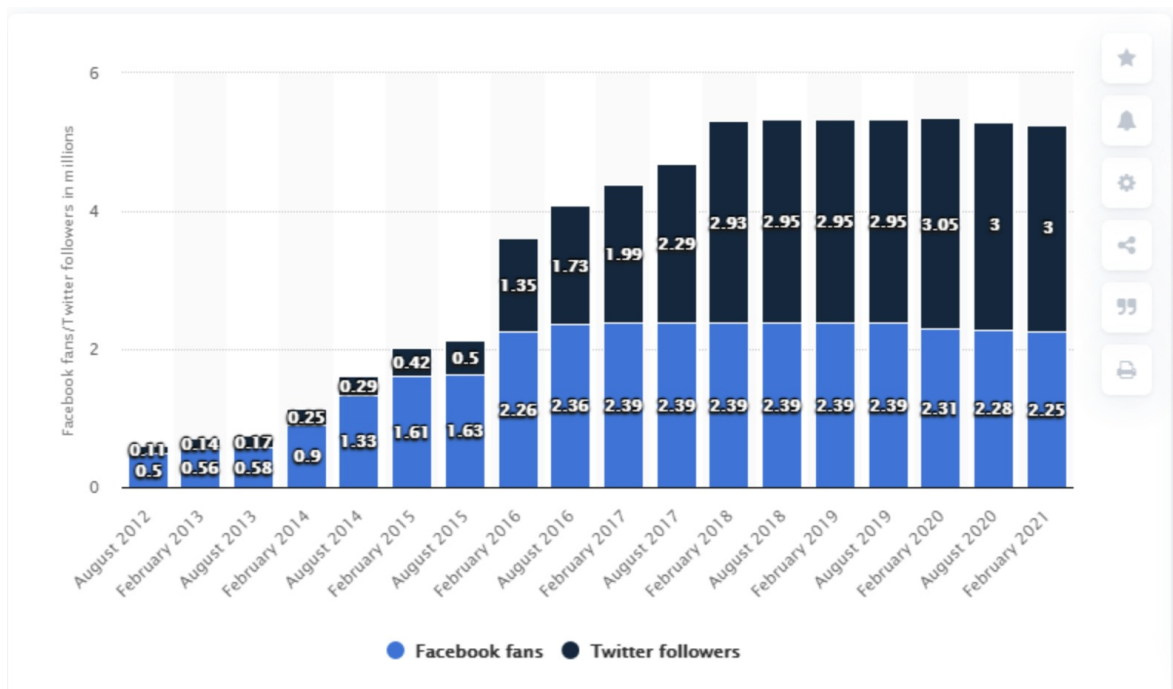
[Cookies Settings](#)

Accept All

Reject All

Establishing a Baseline

I will first evaluate the base GPT-4o model on a dataset with visual question-answer pairs, using Weave Evaluations. The dataset contains questions about charts and tables, along with related visual data, which the model needs to interpret to provide accurate answers. This initial evaluation will help establish a baseline performance for the base GPT-4o model.



Example of the chart in the dataset

After recording the baseline metrics, I will fine-tune the GPT-4o model on a subset of the same dataset. This fine-tuning process will allow the model to learn specific patterns and improve its ability to interpret and respond to questions based on visual inputs.

Once the fine-tuning is complete, I will re-evaluate the fine-tuned model using the same dataset to compare its performance against the baseline. All metrics and evaluations will be tracked using Weights & Biases (W&B), helping to monitor improvements and understand the impact of fine-tuning.

Here's some code to evaluate the base model! Note, you will need to set up your `OPENAI_API_KEY` environment variable before running this script.

```
import base64
import random
import asyncio
```

```

from datasets import load_dataset
from io import BytesIO
from PIL import Image
import os
from openai import OpenAI
import weave
from weave import Evaluation, Model

# Set seed for reproducibility
SEED = 3
random.seed(SEED)

# Initialize Weave evaluation
weave.init('oa_vis_eval')

# Get OpenAI API key from environment variable
api_key = os.getenv("OPENAI_API_KEY")

# Initialize OpenAI client
client = OpenAI(api_key=api_key)

# Load the ChartQA dataset from Hugging Face
dataset = load_dataset("TeeA/ChartQA", split='train', cache_dir="./cache")

# Set the seed for the dataset for reproducibility
dataset = dataset.shuffle(seed=SEED) # Ensure consistent shuffling with the same seed

# Define a helper function to encode a PIL image to base64
def encode_pil_image_to_base64(pil_image):
    buffered = BytesIO()
    pil_image.save(buffered, format="PNG") # Save image as PNG format in memory
    return base64.b64encode(buffered.getvalue()).decode("utf-8")

# Create a subset of the last 30 examples for evaluation
eval_data = dataset.select(range(len(dataset) - 30, len(dataset)))

# Prepare the evaluation dataset
evaluation_dataset = []
for idx in range(len(eval_data)):
    # Retrieve the image from the dataset
    image_path_or_pil = eval_data[idx]["image"]

    # If 'image' is a path, open it. Otherwise, assume it's a PIL Image
    if isinstance(image_path_or_pil, str):
        img = Image.open(image_path_or_pil)

```

```

elif isinstance(image_path_or_pil, Image.Image):
    img = image_path_or_pil
else:
    raise ValueError("Unsupported image type in dataset.")

# Encode the image to base64 format
encoded_image = encode_pil_image_to_base64(img)

# Prepare the evaluation entry with the question, image, and expected answer
entry = {
    "question": eval_data[idx]["qa"][0]["query"],
    "image_base64": f"data:image/png;base64,{encoded_image}",
    "expected": eval_data[idx]["qa"][0]["label"] # Use the 'label' field from 'qa'
}
evaluation_dataset.append(entry)

# Define custom scoring function for Weave evaluation
@weave.op
def substring_match(expected: str, model_output: dict) -> dict:
    match = expected.lower() in model_output['output'].lower()
    return {'substring_match': match}

# Define the base model class
class OpenAIModelBase(Model):
    model_name: str
    model_id: str

    @weave.op()
    def predict(self, question: str, image_base64: str):
        # Create the payload for the OpenAI API request
        messages = [
            {
                "role": "user",
                "content": [
                    {"type": "text", "text": question},
                    {"type": "image_url", "image_url": {"url": image_base64}}
                ]
            }
        ]
        # Perform inference using the OpenAI API
        response = client.chat.completions.create(
            model=self.model_id,
            messages=messages,
            max_tokens=300

```

```

    )

    # Return the model's response for evaluation
    return {"output": response.choices[0].message.content}

# Function to run evaluation
async def run_model_evaluation(model_class, model_name, model_id, evaluation_dataset):
    # Create the evaluation for the model
    evaluation = Evaluation(dataset=evaluation_dataset, scorers=[substring_match])

    # Instantiate the provided model class with the provided name and ID
    model = model_class(model_name=model_name, model_id=model_id)

    # Run the evaluation using the model's predict method
    print(f"Evaluating {model_name} with model ID {model_id}:")
    await evaluation.evaluate(model.predict)
    print(f"{model_name} evaluation completed.")

if __name__ == "__main__":
    # Step 1: Evaluate the base model first
    model_id_base = "gpt-4o-2024-08-06"

    # Check if we're in an environment with an active event loop (e.g., Jupyter/Colab)
    try:
        loop = asyncio.get_running_loop()
    except RuntimeError: # No event loop is running, so we can use asyncio.run
        asyncio.run(run_model_evaluation(OpenAIModelBase, "Base Model", model_id_base, e
    else:
        # If there's an active loop, just await the evaluation directly
        await run_model_evaluation(OpenAIModelBase, "Base Model", model_id_base, evaluat

```

This code begins by evaluating the base GPT-4o model on a subset of visual question-answer data to establish baseline performance metrics. It uses Weave Evaluations from Weights & Biases to organize the evaluation process, making it easier to run comparisons and log results automatically. Weave structures the evaluation by feeding questions and their matching images into the model and tracking how well the predictions match the expected answers.

The code prepares the input by converting images to base64 format so they can be processed by the OpenAI API. It defines a Weave model class that sends the questions and images to the GPT-4o model and returns predictions. A custom scoring function is used within Weave to check if the expected answer is present in the model's output.

The baseline results from this step will be compared against the performance of the fine-tuned model, which will be evaluated later using the same process. If we now open up Weave, we can see our results for the evaluation run in the dashboard. By selecting the evaluations tab, then selecting the run, you will be able to view see more details on the run. Here's a few screenshots of what it will look like inside Weave! Below, we can see a summary of the model's performance, along with traces of each call to the model, so you can dive deeper into the exact responses the model generated!

Metrics	
substring_match:v0	
substring_match.substring_match	0.6667
Model Latency (avg)	8.5266 s
Total Tokens (avg)	24,496

Example 1 of 30	
Input	Value
question	Which product dominates the chart?

Model Outputs	
OpenAIModelBase.predict 5448	
output	The product that dominates the chart is "Manufactures," comprising 77.2% of the total.
Metrics	Trials
substring_match.substring_m	100%
Model Latency (avg)	3.5390s
Total Tokens (avg)	799

Weave also has some really neat evaluation charts, but these are best for comparing multiple models. Stick around, and we will train a custom model, and take full advantage of what Weave has to offer for evaluations!

GPT-4o fine-tuning pricing

Now we will dive into fine-tuning! Training a model through [fine-tuning](#) incurs different costs depending on the model you choose. For example, fine-tuning the [GPT-4o model](#) costs \$25.00 per 1 million training tokens, while GPT-4o-mini is much cheaper, at \$3.00 per 1 million training tokens. In comparison, training tokens for the GPT-3.5 Turbo model are priced at \$8.00 per 1 million tokens. These costs only apply to the tokens consumed during the training process, allowing you to customize the model's behavior and responses according to your specific dataset.

Once a model has been fine-tuned, the inference costs for using that fine-tuned model differ from those of the standard base model. For a fine-tuned `GPT-4o-2024-08-06` model, input tokens cost \$3.75 per 1 million, and output tokens cost \$15.00 per 1 million tokens. This is higher than the **standard** `GPT-4o-2024-08-06` base model, where input tokens are priced at \$2.50 per 1 million and output tokens at \$10.00 per 1 million

For the fine-tuned GPT-4o-mini model, input tokens are billed at \$0.30 per 1 million input tokens, while output tokens cost \$1.20 per 1 million. These are higher than the standard

GPT-4o-mini rates, where input tokens cost \$0.15 per 1 million and output tokens are \$0.60 per 1 million. If you use the Batch API with the fine-tuned model, the rates are reduced to \$0.15 per 1 million for input tokens and \$0.60 per 1 million for output tokens, compared to \$0.075 and \$0.30 per 1 million tokens for the standard base model.

According to OpenAI, most images processed using the low detail setting cost a fixed 85 tokens per image, regardless of size or complexity. This is the standard cost for low-detail images and provides predictable pricing. For images processed at a high detail setting, the cost varies based on the image size. The image is resized to fit within a 2048 x 2048 square and divided into 512 x 512 pixel tiles, with each tile costing 170 tokens. An additional 85 tokens are always added to the total. For example, a 1024 x 1024 image in high detail mode would cost 765 tokens, while a larger 2048 x 4096 image would require more tiles and cost 1105 tokens.

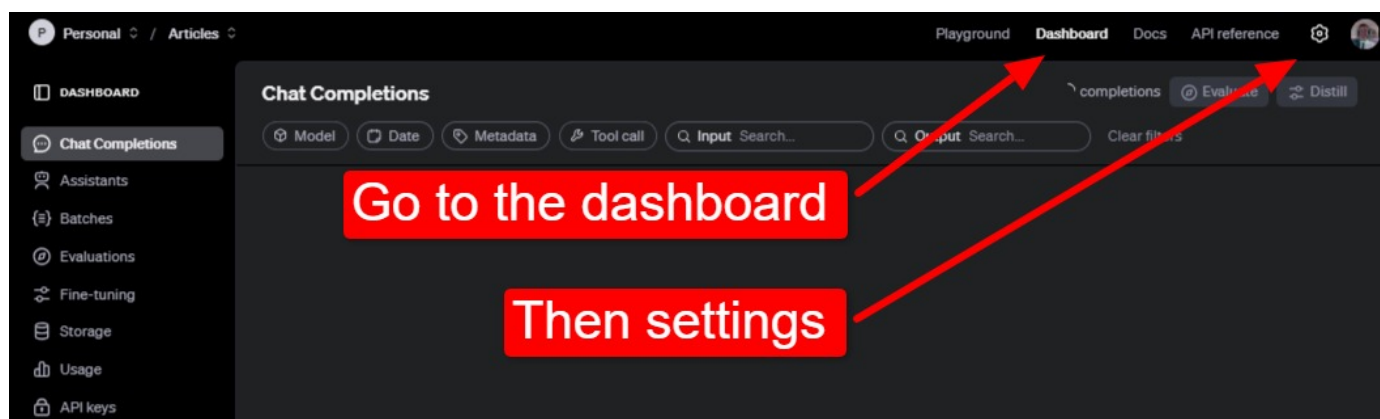
Environment setup for vision fine-tuning GPT-4o

Setting up your OpenAI and Weights & Biases integration is crucial for managing the fine-tuning of a model like GPT-4o and tracking model performance effectively. First, obtain your OpenAI API key, which is required for authenticating requests to OpenAI's services. Ensure that this key is securely stored and set as an environment variable, such as `OPENAI_API_KEY`, in your development environment. This will allow you to perform tasks like model fine-tuning and inference.

You can set the key using the following command:

```
export OPENAI_API_KEY="your_api_key"
```

Next, navigate to the OpenAI dashboard and access your organization settings.

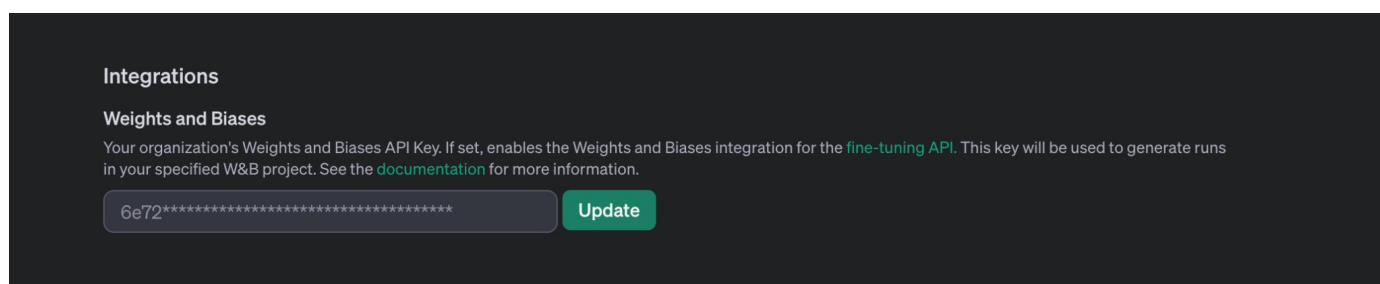


You will see the Organization ID, which you can copy for reference if needed. The Organization ID is sometimes used in API requests or when integrating additional

services. After locating your Organization ID, scroll down to the Integrations section, where you will find an option to add your Weights & Biases API key. Click the Update button and paste your API key in the provided field.

💡 You can find your Weights & Biases API key at <https://wandb.ai/authorize>.

This step enables the integration between OpenAI and Weights & Biases, allowing you to track and visualize your fine-tuning jobs in your specified W&B project.

A screenshot of the OpenAI dashboard's 'Integrations' section. Under the 'Weights and Biases' heading, there is a text description: 'Your organization's Weights and Biases API Key. If set, enables the Weights and Biases integration for the fine-tuning API. This key will be used to generate runs in your specified W&B project. See the documentation for more information.' Below this text is a text input field containing the API key '6e72*****' and a green 'Update' button to its right.

Ensure that the integration is enabled and visible in the OpenAI dashboard. You should see a confirmation indicating that the integration is active. After successfully adding the Weights & Biases API key, log in using the CLI command `wandb login`. This ensures that your environment has the necessary credentials to run fine-tuning jobs and log data efficiently.

With these configurations in place, your OpenAI and Weights & Biases accounts are now linked, making it easier to run and monitor fine-tuning jobs. This integration allows you to track metrics and performance in real-time, making it a powerful setup for model evaluation and iteration.

Before vision fine-tuning and evaluating our GPT-4o model, make sure you have all the necessary libraries installed. You will need Weave, OpenAI, Pillow, and Datasets. These libraries provide essential tools for managing and processing datasets, running fine-tuning jobs, and evaluating models. Open a terminal and run the following command:

```
pip install weave openai pillow datasets
```

Weave is used for tracking, visualizing, and evaluating your models, while the OpenAI package is necessary for interacting with OpenAI's API for tasks like vision fine-tuning GPT-4o and inference. Pillow is required for image processing, such as converting and encoding images into formats compatible with the API. The Datasets library by [HuggingFace](#) is useful for loading and manipulating the ChartQA dataset or any other dataset you want to use for fine-tuning your model.

Vision fine-tuning GPT-4o

Vision fine-tuning a model like GPT-4o allows you to adapt its behavior based on your unique dataset. This process involves providing labeled training data so the model can learn specific patterns, contexts, and nuances available in your data that is not fully captured by the base model. Fine-tuning is especially useful when dealing with specialized tasks, such as interpreting visual data, domain-specific language, or any context-specific scenarios.

To start off, we will need to prepare our dataset and upload it to OpenAI's servers. The code for fine-tuning is relatively concise, so I will include the data preparation and fine-tuning code all in one script.

Now we will write a script to fine-tune the GPT-4o model using the ChartQA dataset, which is designed to train models for visual question answering based on charts and tables. The dataset provides question-answer pairs that are linked to specific charts, allowing the model to learn how to interpret and respond to questions derived from visual data.

```
import base64
import json
from datasets import load_dataset
from openai import OpenAI
from io import BytesIO
from PIL import Image
import matplotlib.pyplot as plt
import os

api_key = os.getenv('OPENAI_API_KEY')
# Initialize the OpenAI client with your API key
client = OpenAI(api_key=api_key)

# Load the ChartQA dataset
dataset = load_dataset("TeeA/ChartQA", split='train', cache_dir="./cache")

# Define a helper function to convert PIL image to base64
def encode_pil_image_to_base64(pil_image):
    buffered = BytesIO()
    pil_image.save(buffered, format="PNG") # Save image as PNG format in memory
    return base64.b64encode(buffered.getvalue()).decode("utf-8")

# Prepare training data in the required JSONL format
training_data = dataset.select(range(100)) # Select the first 100 examples for training
validation_data = dataset.select(range(100, 120)) # Select 20 examples for validation
```

```

training_jsonl = []
for idx, example in enumerate(training_data):
    # Assuming 'image' column contains paths or PIL images. Adjust based on dataset structure
    image_path_or_pil = example["image"]

    # If 'image' is a path, open it. Otherwise, assume it's a PIL Image
    if isinstance(image_path_or_pil, str):
        img = Image.open(image_path_or_pil)
    elif isinstance(image_path_or_pil, Image.Image):
        img = image_path_or_pil
    else:
        raise ValueError("Unsupported image type in dataset.")

    # Encode the image to base64
    encoded_image = encode_pil_image_to_base64(img)

    # Prepare JSONL entry with question and answer fields
    entry = {
        "messages": [
            {"role": "system", "content": "You are an assistant that processes charts and answers questions about them."},
            {"role": "user", "content": example["qa"][0]["query"]}, # Use 'query' field for user question
            {
                "role": "user",
                "content": [
                    {"type": "image_url", "image_url": {"url": f"data:image/png;base64,{encoded_image}"}}
                ]
            },
            {"role": "assistant", "content": example["qa"][0]["label"]} # Use 'label' field for assistant answer
        ]
    }

    training_jsonl.append(json.dumps(entry)) # Use json.dumps to create a valid JSON string

# Save training data to JSONL file
train_filename = "chartqa_training_data.jsonl"
with open(train_filename, "w") as train_file:
    for entry in training_jsonl:
        train_file.write(f"{entry}\n")

# Upload training file
train_file = client.files.create(
    file=open(train_filename, "rb"),
    purpose="fine-tune"
)

```

```

)
training_file_id = train_file.id

# Repeat the process for validation data
validation_jsonl = []
for idx, example in enumerate(validation_data):
    # Assuming 'image' column contains paths or PIL images. Adjust based on dataset structure
    image_path_or_pil = example["image"]

    # If 'image' is a path, open it. Otherwise, assume it's a PIL Image
    if isinstance(image_path_or_pil, str):
        img = Image.open(image_path_or_pil)
    elif isinstance(image_path_or_pil, Image.Image):
        img = image_path_or_pil
    else:
        raise ValueError("Unsupported image type in dataset.")

    # Encode the image to base64
    encoded_image = encode_pil_image_to_base64(img)

    # Prepare JSONL entry with question and answer fields
    entry = {
        "messages": [
            {"role": "system", "content": "You are an assistant that processes charts and answers questions."},
            {"role": "user", "content": example["qa"][0]["query"]}, # Use 'query' field for question
            {
                "role": "user",
                "content": [
                    {"type": "image_url", "image_url": {"url": f"data:image/png;base64,{encoded_image}"}}
                ]
            },
            {"role": "assistant", "content": example["qa"][0]["label"]} # Use 'label' field for answer
        ]
    }

    validation_jsonl.append(json.dumps(entry)) # Use json.dumps to create a valid JSONL entry

# Save validation data to JSONL file
val_filename = "chartqa_validation_data.jsonl"
with open(val_filename, "w") as val_file:
    for entry in validation_jsonl:
        val_file.write(f"{entry}\n")

# Upload validation file
val_file = client.files.create(

```

```

        file=open(val_filename, "rb"),
        purpose="fine-tune"
    )
    validation_file_id = val_file.id

# Create a fine-tuning job with the uploaded training and validation files
response = client.fine_tuning.jobs.create(
    training_file=training_file_id,
    validation_file=validation_file_id,
    model="gpt-4o-2024-08-06",
    integrations=[
        {
            "type": "wandb",
            "wandb": {
                "project": "chartqa-finetuning",
                "tags": ["charts", "qa", "base64-images"]
            }
        }
    ]
)
print(f"Created fine-tuning job: {response.id}")

```

The script begins by loading the ChartQA dataset using the `datasets` library, selecting a subset for training and validation. Each entry in the dataset contains a question, a corresponding chart or table image, and an expected answer. *The script processes each example by converting the images into base64 format and structuring them into a JSONL format that is compatible with OpenAI's API.*

This JSONL format includes a system message to set the context, a user message that combines the question and the encoded image, and an assistant message with the expected answer. This format ensures that the model has all the necessary context for generating the desired output during fine-tuning. Here's an example of a JSONL dataset with one example, omitting the full base64 encoded string.

```

{"messages": [{"role": "system", "content": "You are an assistant that processes charts"}

```

After preparing the dataset, the script uploads the training and validation files to OpenAI. A GPT-4o fine-tuning job is then created using these files, specifying the `gpt-4o-2024-08-06` model. The script also integrates with Weights & Biases to log and track the fine-tuning progress, making it easier to visualize the training metrics and monitor performance.

By running this script, you can vision fine-tune the base model on the ChartQA dataset,

enabling it to better handle complex queries related to visual data. This process not only helps the model generate more accurate responses but also ensures that it performs effectively on tasks involving the interpretation of charts and tables.

Once the fine-tuning job is initiated, its progress and results will automatically be tracked and visualized in your Weights & Biases workspace. OpenAI's integration with Weights & Biases provides a clear view of several metrics, including training loss, validation loss, and token accuracy over time.

In the Weights & Biases dashboard, each run is represented as a distinct entry under the Runs section. You can see the detailed logs and metrics in the form of charts and graphs that update in real-time as the vision fine-tuning progresses. For example, you can observe how the training loss decreases as the model learns from the training data and how validation loss behaves on the validation set. Here are the charts for my run:

✓ Run: ftjob-8aDxCgFL7itQtBwT4IRmhnuB 1 ⋮

This real-time tracking and visualization enable you to understand how the model is evolving and make adjustments as needed. With the integration in place, Weights & Biases becomes a powerful tool for monitoring and improving your model's fine-tuning process, providing insights that are crucial for optimizing performance.

Evaluating our vision fine-tuned GPT-4o model

We would now like to compare the performance of our fine-tuned model against the base model to evaluate the improvements made during vision fine-tuning. We will use [Weave Evaluations](#), which is a framework built by Weights & Biases and allows us to quickly evaluate our fine-tuned GPT-4o model, and also visualize the performance for specific examples in the evaluation dataset. The script below helps us achieve this by defining two separate classes—one for the fine-tuned model and one for the base model—and running an evaluation on both using the ChartQA dataset.

```
import base64
import random
import json
import asyncio
from datasets import load_dataset
from io import BytesIO
from PIL import Image
import os
from openai import OpenAI
import weave
from weave import Evaluation, Model

# Set seed for reproducibility
SEED = 3
random.seed(SEED)

# Initialize Weave evaluation
weave.init('oa_vis_eval')

# Get OpenAI API key from environment variable
```

```

api_key = os.getenv("OPENAI_API_KEY")

# Initialize OpenAI client
client = OpenAI(api_key=api_key)

# Load the ChartQA dataset from Hugging Face
dataset = load_dataset("TeeA/ChartQA", split='train', cache_dir="./cache")

# Set the seed for the dataset for reproducibility
dataset = dataset.shuffle(seed=SEED) # Ensure consistent shuffling with the same seed

# Define a helper function to encode a PIL image to base64
def encode_pil_image_to_base64(pil_image):
    buffered = BytesIO()
    pil_image.save(buffered, format="PNG") # Save image as PNG format in memory
    return base64.b64encode(buffered.getvalue()).decode("utf-8")

# Define the function to sample an image and query from the dataset
def sample_image_and_query_from_dataset(dataset, index):
    example = dataset[index]

    # Retrieve the image from the dataset
    image_path_or_pil = example["image"]

    # If 'image' is a path, open it. Otherwise, assume it's a PIL Image
    if isinstance(image_path_or_pil, str):
        img = Image.open(image_path_or_pil)
    elif isinstance(image_path_or_pil, Image.Image):
        img = image_path_or_pil
    else:
        raise ValueError("Unsupported image type in dataset.")

    # Encode the image to base64 format
    encoded_image = encode_pil_image_to_base64(img)

    # Retrieve the corresponding query
    query = example["qa"][0]["query"]

    return query, encoded_image

# Create a subset of the last 30 examples for evaluation
eval_data = dataset.select(range(len(dataset) - 30, len(dataset)))

# Prepare the evaluation dataset
evaluation_dataset = []

```

```

for idx in range(len(eval_data)):
    # Get the query and encoded image
    query, encoded_image = sample_image_and_query_from_dataset(eval_data, idx)

    # Prepare the evaluation entry with the question, image, and expected answer
    entry = {
        "question": query,
        "image_base64": f"data:image/png;base64,{encoded_image}",
        "expected": eval_data[idx]["qa"][0]["label"] # Use the 'label' field from 'qa'
    }
    evaluation_dataset.append(entry)

# Define custom scoring function for Weave evaluation
@weave.op()
def substring_match(expected: str, model_output: dict) -> dict:
    match = expected.lower() in model_output['output'].lower()
    return {'substring_match': match}

class OpenAIModelFT(Model):
    model_name: str
    model_id: str

    @weave.op()
    def predict(self, question: str, image_base64: str):
        # Create the payload for the OpenAI API request
        messages = [
            {
                "role": "user",
                "content": [
                    {"type": "text", "text": question},
                    {"type": "image_url", "image_url": {"url": image_base64}}
                ]
            }
        ]

        # Perform inference using the OpenAI API
        response = client.chat.completions.create(
            model=self.model_id,
            messages=messages,
            max_tokens=300
        )

```

```

        # Return the model's response for evaluation
        return {"output": response.choices[0].message.content}

# Define the stock model class for evaluation
class OpenAIModelStock(Model):
    model_name: str
    model_id: str

    @weave.op()
    def predict(self, question: str, image_base64: str):
        # Create the payload for the OpenAI API request
        messages = [
            {
                "role": "user",
                "content": [
                    {"type": "text", "text": question},
                    {"type": "image_url", "image_url": {"url": image_base64}}
                ]
            }
        ]

        # Perform inference using the OpenAI API
        response = client.chat.completions.create(
            model=self.model_id,
            messages=messages,
            max_tokens=300
        )

        # Return the model's response for evaluation
        return {"output": response.choices[0].message.content}

# Function to run evaluation for a given model instance
async def run_model_evaluation(model_class, model_name, model_id, evaluation_dataset):
    # Create the evaluation for the model
    evaluation = Evaluation(dataset=evaluation_dataset, scorers=[substring_match])

    # Instantiate the provided model class with the provided name and ID
    model = model_class(model_name=model_name, model_id=model_id)

    # Run the evaluation using the model's predict method
    print(f"Evaluating {model_name} with model ID {model_id}:")
    await evaluation.evaluate(model.predict) # Use await directly
    print(f"{model_name} evaluation completed.")

```

```

if __name__ == "__main__":
    # Define the model IDs for both models
    model_id_finetuned = "ft:gpt-4o-2024-08-06:3dsports::AEZLp5TQ"
    model_id_base = "gpt-4o-2024-08-06"

    # Check if an event loop is already running (Colab or Jupyter)
    try:
        loop = asyncio.get_running_loop()
    except RuntimeError:
        # No event loop, so we can use asyncio.run
        asyncio.run(run_model_evaluation(OpenAIModelFT, "OpenAI_4o_vision_ft", model_id_finetuned))
        asyncio.run(run_model_evaluation(OpenAIModelStock, "OpenAI_4o_vision_base", model_id_base))
    else:
        # If there's an active loop, just await the evaluation directly
        await run_model_evaluation(OpenAIModelFT, "OpenAI_4o_vision_ft", model_id_finetuned)
        await run_model_evaluation(OpenAIModelStock, "OpenAI_4o_vision_base", model_id_base)

```

The script first prepares a subset of evaluation data by selecting the last 20 examples from the shuffled dataset and encoding each chart image in base64 format. This ensures that both models are tested on the same set of inputs for consistency. The `OpenAIModelFT` class represents the fine-tuned model, while the `OpenAIModelStock` class represents the base model without any additional training. Each class includes a `predict` method that sends a query and corresponding chart image to the specified model through the OpenAI API, returning the model's response.

The `run_model_evaluation` function is used to evaluate each model on the same dataset. It instantiates the specified model class, passes it to a `weave.Evaluation` instance, and runs the evaluation using the model's `predict` method. This allows us to measure the performance of both models using a custom scoring function, `substring_match`, which checks if the expected answer is a substring of the model's output. This isn't the comprehensive metric, however for our purposes, it works fairly well.

After running this script, you will be able to visualize the results in the Weave comparisons dashboard. This dashboard provides a side-by-side comparison of key metrics like token accuracy and accuracy for both models, making it easy to analyze their performance. By selecting the evaluation runs in the Weights & Biases project and clicking on the 'Compare' button, you can generate detailed visualizations that highlight the differences between the vision fine-tuned and base GPT-4o models. This helps you gain insights into areas where the fine-tuned model performs better and where further improvements can be made.

● Evaluation bc25

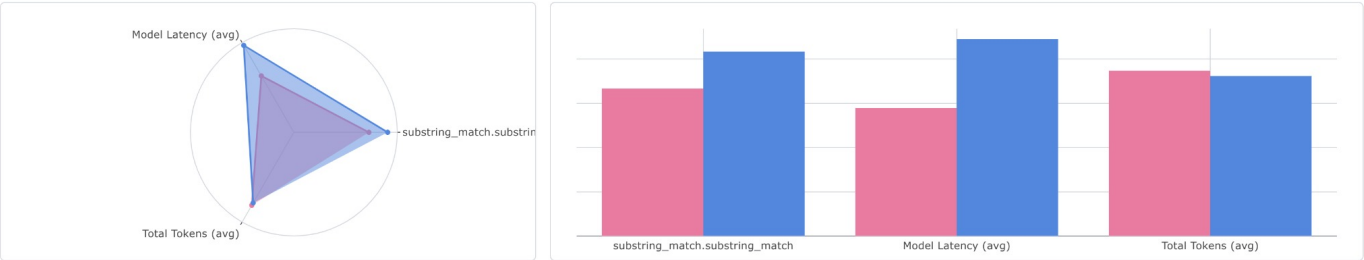
⚙ OpenAIModelStock.predict:v4

↻

● Evaluation e9f4

⚙ OpenAIModelFT.predict:v4

Summary Metrics



Scorecard

Evaluation		Evaluation bc25		Evaluation e9f4	
Model	OpenAIModelStock.predict:v4		OpenAIModelFT.predict:v4		
Properties					
Metrics					
substring_match:v0					
substring_match.substring_match	0.6667		0.8333	+0.1667	
Model Latency (avg)	18.4919 s		28.4816 s	+9.9897s	
Total Tokens (avg)	24,476		23,690	-786	

The comparison dashboard shows the evaluation results for our fine-tuned model and the base model on the same set of 30 examples from the ChartQA dataset. Because we only fine-tuned on a small number of examples, the overall performance metrics between the two models are relatively similar. This indicates that the fine-tuning process has not yet captured the full potential improvement that could be achieved with a larger training set. Below show's the compare view for each individual sample in the dataset!

Output Comparison

📄 Example 1 of 30

Input	Value
question	What was the value of the Tampa Bay Buccaneers in dollars in 2020?

Model Outputs		OpenAIModelStock.predict ae46		OpenAIModelFT.predict 7457	
output		In 2020, the value of the Tampa Bay Buccaneers was \$2,280 million (or \$2.28 billion).		2280	
Metrics		Trials		Trials	
		👁 0		👁 0	
substring_match.substring_m	0%	✗		✓	
Model Latency (avg)	2.5540s	2.5540		24.9200s	
Total Tokens (avg)	816	816		791	

The comparison dashboard highlights the evaluation results of our fine-tuned model against the base model on the same set of 30 examples from the ChartQA dataset. Our fine-tuned model achieved a **16.16%** improvement in the `substring_match` metric compared to the base model. This indicates that fine-tuning, even on a limited set of examples, can significantly enhance model performance and better capture the nuances in the data.

Overall

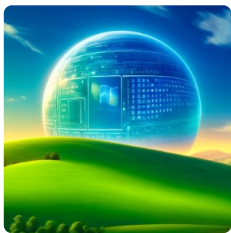
In conclusion, [fine-tuning an OpenAI model](#) on the ChartQA dataset highlighted how specialized datasets can significantly improve a model's ability to interpret visual data. The integration with Weights & Biases and W&B Weave provided real-time tracking, performance evaluation, and visualization, making the fine-tuning process more transparent and efficient.

By leveraging these tools, you can optimize and evaluate model performance effectively, enabling the development of highly specialized AI solutions for complex tasks. I hope you enjoyed this tutorial and found it valuable for your AI fine-tuning journey. Happy experimenting and fine-tuning! Here's a [link](#) to a Google Colab for the project.



How to train and evaluate an LLM router

This tutorial explores LLM routers, inspired by the RouteLLM paper, covering training, evaluation, and practical use cases for managing LLMs ...



How to fine-tune Phi-3 Vision on a custom dataset

Here's how to fine tune a state of the art multimodal LLM on a custom dataset



How to Fine-Tune LLaVA on a Custom Dataset

A tutorial for fine-tuning LLaVA on your own data!



Skin Lesion Classification on HAM10000 with HuggingFace using PyTorch and W&B

Explore the use of HuggingFace, PyTorch, and W&B for classifying skin lesions with the HAM10000 dataset. We will build, train, and evaluate ...

Add a comment

Tags: Articles, OpenAI, GenAI, Computer Vision, Tutorial

New course
LLM Apps: Evaluation

ENROLL NOW →

Weights
& Biases

All Hands

Google

Made with Weights & Biases. [Sign up](#) or [log in](#) to create reports like this one.

Never lose track of
another ML project.
Try **Weights & Biases** today.

SIGN UP

TRY W&B NOW



Weights & Biases

Get weekly updates with the latest ML news.

Subscribe

PRODUCTS

[Dashboard](#) | [Sweeps](#) | [Artifacts](#) | [Reports](#) | [Tables](#)

QUICKSTART

[Documentation](#)

RESOURCES

[Courses](#) | [Forum](#) | [Tutorials](#) | [Benchmarks](#)

W&B

[About Us](#) | [Authors](#) | [Contact](#) | [Terms of Service](#) | [Privacy Policy](#)

Copyright ©2025 Weights & Biases. All rights reserved.